ARRAY VS LINKED LIST

Array and **Linked list** are the two ways of organizing the data in the memory.

What is an array?

- An array is a data structure that contains the elements of the same type.
- A data structure is a way of organizing the data; an array is a data structure because it sequentially organizes the data.
- An array is a big chunk of memory in which memory is divided into small-small blocks, and each block is capable of storing some value.
- Suppose we have created an array that consists of 10 values, then each block will store the value of an integer type.
- If we try to store the value in an array of different types, then it is not a correct array and will throw a compile-time error.

Declaration of array

- An array can be declared as:

  data_type name of the array[no of elements]

- To declare an array, we first need to specify the type of the array and then the array's name.
- Inside the square brackets, we need to specify the number of elements that our array should contain.
- Let's understand through an example.

  int a[5];

- In the above case, we have declared an array of 5 elements with 'a' name of an integer data type.

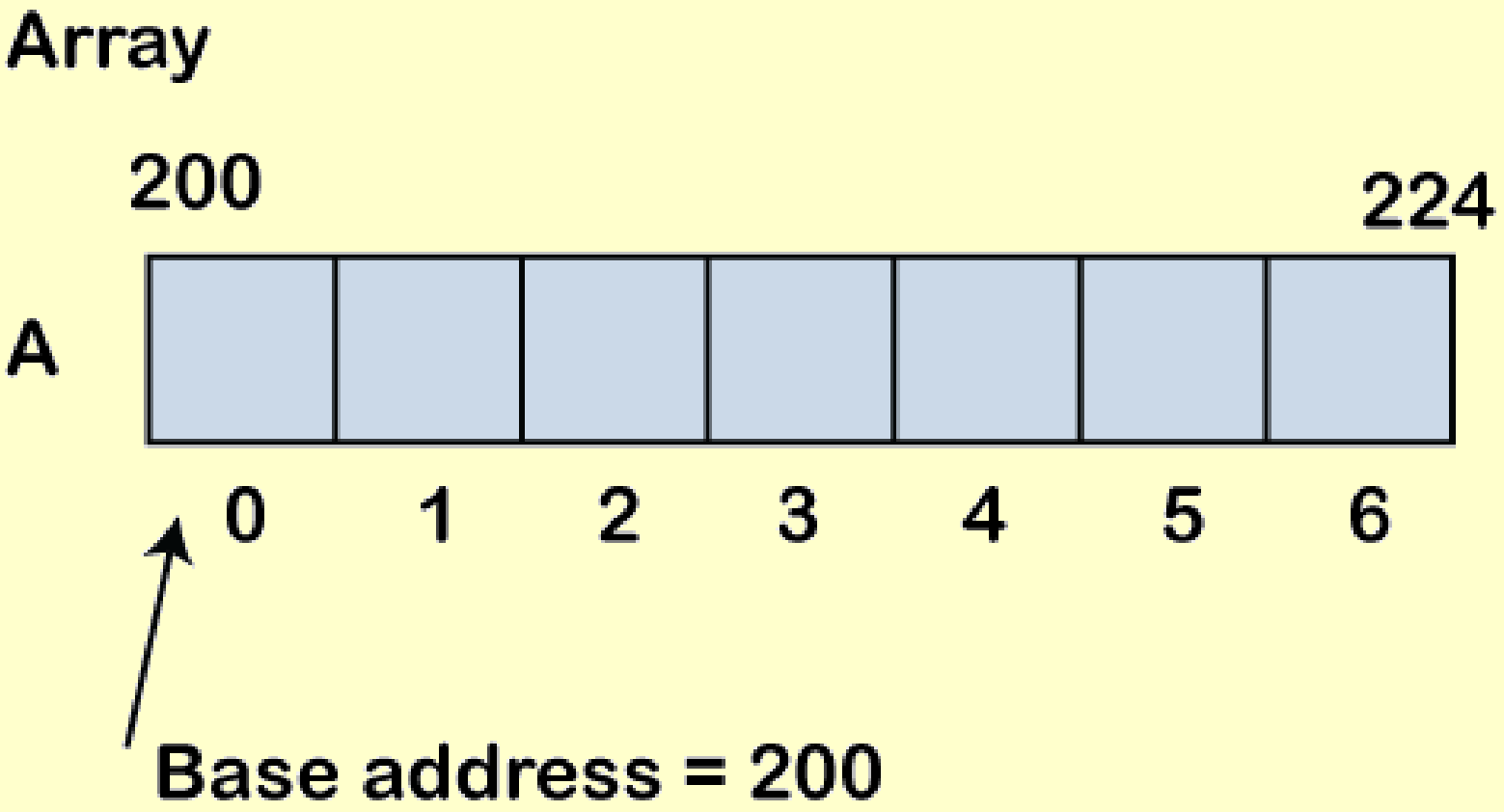
What is Linked list?

- A linked list is the collection of nodes that are randomly stored.
- Each node consists of two fields, i.e., **data** and **link.**
- Here, data is the value stored at that particular node, and the link is the pointer that holds the address of the next node.

DIFFERENCES BETWEEN ARRAY AND LINKED LIST

1. Cost of accessing an element

- In case of an array, irrespective of the size of an array, an array takes a constant time for accessing an element.
- In an array, the elements are stored in a contiguous manner, so if we know the base address of the element, then we can easily get the address of any element in an array.
- We need to perform a simple calculation to obtain the address of any element in an array.
- So, accessing the element in an array is **O(1)** in terms of time complexity.


Array
200
224
A
0
1
2
3
4
5
6
Base address = 200

- In the linked list, the elements are not stored in a contiguous manner.
- It consists of multiple blocks, and each block is represented as a node.
- Each node has two fields, i.e., one is for the data field, and another one stores the address of the next node.
- To find any node in the linked list, we first need to determine the first node known as the head node.
- If we have to find the second node in the list, then we need to traverse from the first node, and in the worst case, to find the last node, we will be traversing all the nodes.
- The average case for accessing the element is O(n).

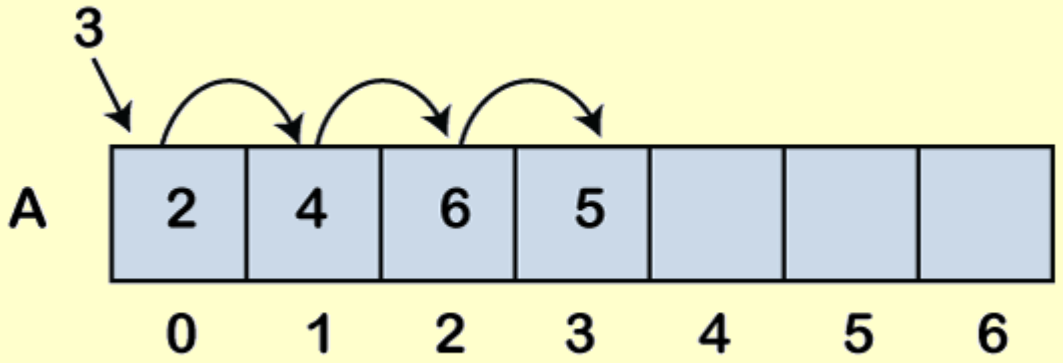
- We conclude that the cost of accessing an element in array is less than the linked list.
- Therefore, if we have any requirement for accessing the elements, then array is a better choice.

2. Cost of inserting an element

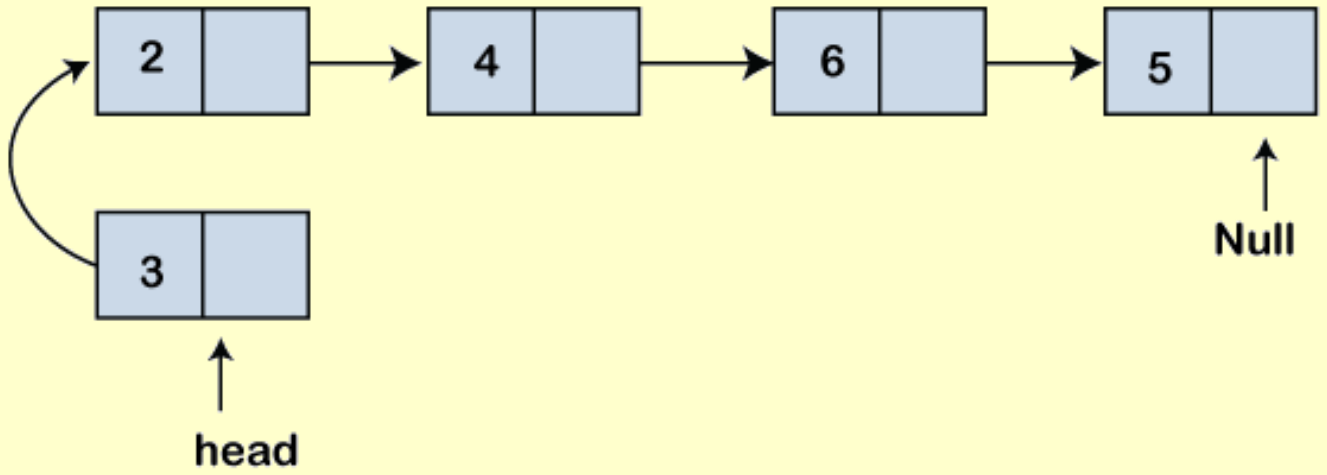
There can be three scenarios in the insertion:

Inserting the element at the beginning:

- To insert the new element at the beginning, we first need to shift the element towards the right to create a space in the first position.
- So, the time complexity will be proportional to the size of the list.
- If n is the size of the array, the time complexity would be O(n).



- In the case of a linked list, to insert an element at the starting of the linked list, we will create a new node, and the address of the first node is added to the new node.
- In this way, the new node becomes the first node. So, the time complexity is not proportional to the size of the list.
- The time complexity would be constant, i.e., O(1).

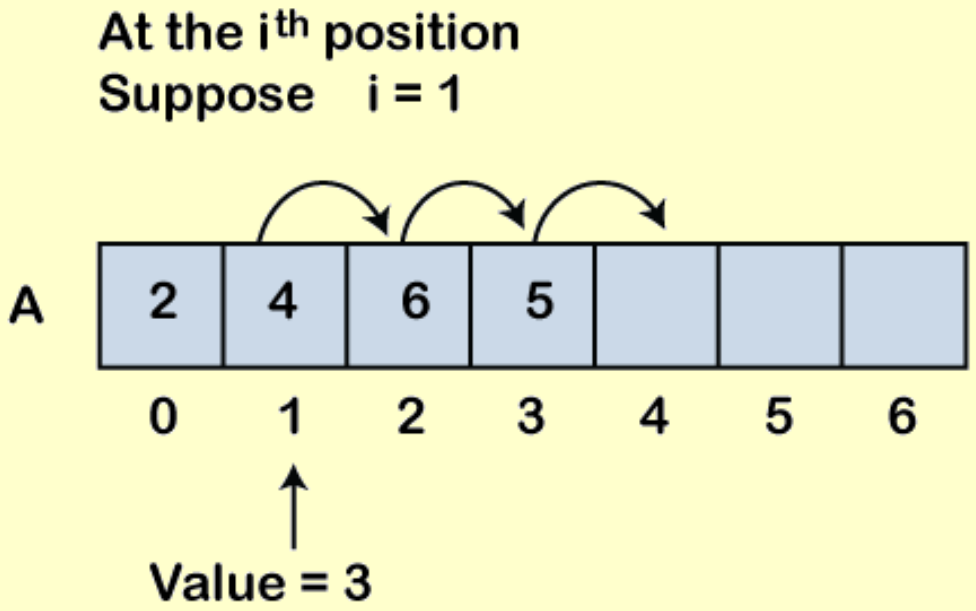


Inserting an element at the end

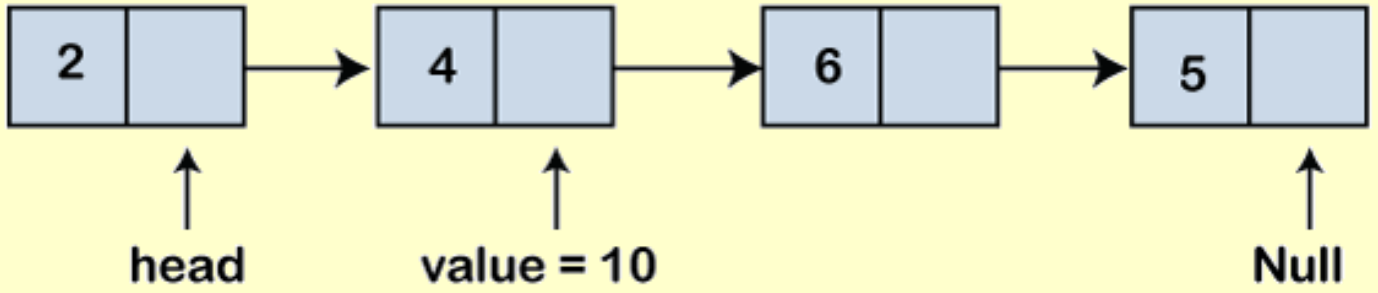
- If the array is not full, then we can directly add the new element through the index. In this case, the time complexity would be constant, i.e., O(1).
- If the array is full, we first need to copy the array into another array and add a new element.
- In this case, the time complexity would be O(n).
- To insert an element at the end of the linked list, we have to traverse the whole list. If the linked list consists of n elements, then the time complexity would be O(n).

Inserting an element at the mid

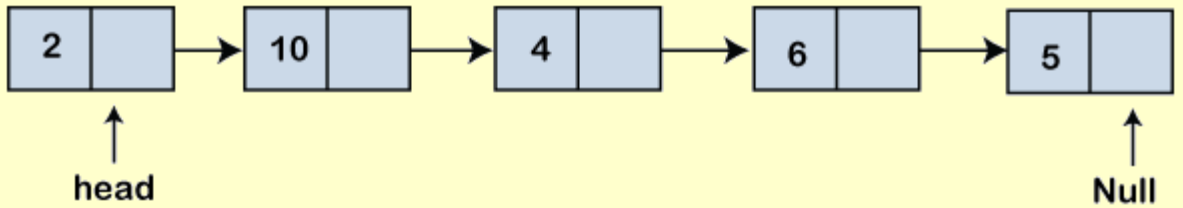
- Suppose we want to insert the element at the ith position of the array; we need to shift the n/2 elements towards the right.
- Therefore, the time complexity is proportional to the number of the elements.
- The time complexity would be O(n) for the average case.



- In the case of linked list, we have to traverse to that position where we have to insert the new element.
- Even though, we do not have to perform any kind of shifting, but we have to traverse to n/2 position.
- The time taken is proportional to the n number of elements, and the time complexity for the average case would be O(n).



- The resultant linked list is:



Ease of use

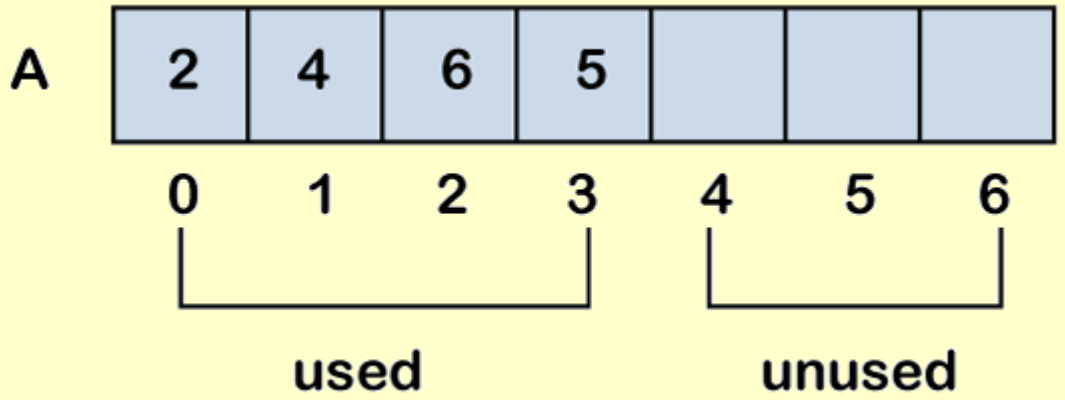
- The implementation of an array is easy as compared to the linked list.
- While creating a program using a linked list, the program is more prone to errors like segmentation fault or memory leak.
- So, lots of care need to be taken while creating a program in the linked list.

Dynamic in size

- The linked list is dynamic in size whereas the array is static.
- Here, static doesn't mean that we cannot decide the size at the run time, but we cannot change it once the size is decided.

3. Memory requirements

- As the elements in an array store in one contiguous block of memory, so array is of fixed size.
- Suppose we have an array of size 7, and the array consists of 4 elements then the rest of the space is unused.
- The memory occupied by the 7 elements:
- **Memory space = 7*4 = 28 bytes**
- Where 7 is the number of elements in an array and 4 is the number of bytes of an integer type.



- In case of linked list, there is no unused memory but the extra memory is occupied by the pointer variables.
- If the data is of integer type, then total memory occupied by one node is 8 bytes, i.e., 4 bytes for data and 4 bytes for pointer variable.
- If the linked list consists of 4 elements, then the memory space occupied by the linked list would be:
- **Memory space = 8*4 = 32 bytes**

- The linked list would be a better choice if the data part is larger in size.
- Suppose the data is of 16 bytes.
- The memory space occupied by the array would be 16*7=112 bytes while the linked list occupies 20*4=80, here we have specified 20 bytes as 16 bytes for the size of the data plus 4 bytes for the pointer variable.
- If we are choosing the larger size of data, then the linked list would consume a less memory; otherwise, it depends on the factors that we are adopting to determine the size.

Array	Linked list
An array is a collection of elements of a similar data type.	A linked list is a collection of objects known as a node where node consists of two parts, i.e., data and address.
Array elements store in a contiguous memory location.	Linked list elements can be stored anywhere in the memory or randomly stored.
Array works with a static memory. Here static memory means that the memory size is fixed and cannot be changed at the run time.	The Linked list works with dynamic memory. Here, dynamic memory means that the memory size can be changed at the run time according to our requirements.
Array elements are independent of each other.	Linked list elements are dependent on each other. As each node contains the address of the next node so to access the next node, we need to access its previous node.

Array	Linked list
Array takes more time while performing any operation like insertion, deletion, etc.	Linked list takes less time while performing any operation like insertion, deletion, etc.
Accessing any element in an array is faster as the element in an array can be directly accessed through the index.	Accessing an element in a linked list is slower as it starts traversing from the first element of the linked list.
In the case of an array, memory is allocated at compile-time.	In the case of a linked list, memory is allocated at run time.
Memory utilization is inefficient in the array. For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused.	Memory utilization is efficient in the case of a linked list as the memory can be allocated or deallocated at the run time according to our requirement.

DYNAMIC STORAGE MANAGEMENT

- Static memory allocation is simple and convenient, but it's not sufficient for everything.
- Two basic operations in dynamic storage management:
  - Allocate a given number of bytes
  - Free a previously allocated block
- Two general approaches to dynamic storage allocation:
  - Stack allocation (hierarchical): restricted, but simple and efficient.
  - Heap allocation: more general, but less efficient, more difficult to implement.

Stack Allocation

- A stack can be used when memory allocation and freeing are partially predictable: memory is freed in opposite order from allocation.
- If alloc(A) then alloc(B) then alloc(C), then it will be free(C) then free(B) then free(A).
- Example: procedure call. X calls Y calls Y again.
- Stacks are also useful for lots of other things: tree traversal, expression evaluation, top-down recursive descent parsers, etc.
- A stack-based organization keeps all the free space together in one place.

Heap Allocation

- Heap allocation must be used when allocation and release are unpredictable
- Memory consists of allocated areas and free areas (or holes). Inevitably end up with lots of holes.
- Goal: reuse the space in holes to keep the number of holes small, keep their size large.
- *Fragmentation*: inefficient use of memory due to holes that are too small to be useful. Stack allocation is perfect: only one hole.
- Typically, heap allocation schemes use a *free list* to keep track of the storage that is not in use.

- *Best fit*: keep linked list of free blocks, search the whole list on each allocation, choose block that comes closest to matching the needs of the allocation, save the excess for later. During release operations, merge adjacent free blocks.
- *First fit*: just scan list for the first hole that is large enough. Free excess. Also merge on releases. Most first fit implementations are rotating first fit.
- Best fit is not necessarily better than first fit!
- First fit tends to leave "average" size holes, while best fit tends to leave some very large ones, some very small ones. The very small ones can't be used very easily.

- Problem: over time, holes tend to fragment, approaching the size of the smallest objects allocated
- *Bit map*: used for managing storage that comes in fixed-size chunks (e.g. disk blocks, or 32-byte chunks).
  - Keep a large array of bits, one for each chunk.
  - If bit is 0 it means chunk is in use, if bit is 1 it means chunk is free.
- *Pools*: keep a separate linked list for each popular size.
  - Allocation is fast, no fragmentation.

Storage Reclamation

- How do we know when dynamically-allocated memory can be freed?
  - Easy when a chunk is only used in one place.
  - Reclamation is hard when information is shared: it can't be recycled until all of the users are finished.
  - Usage is indicated by the presence of *pointers* to the data. Without a pointer, can't access (can't find it).
- Two problems in reclamation:
  - Dangling pointers: better not recycle storage while it's still being used.
  - Memory leaks: storage gets "lost" because no one freed it even though it can't ever be used again.

- *Reference counts*: keep count of the number of outstanding pointers to each chunk of memory. When this becomes zero, free the memory. Example: Smalltalk, file descriptors in Unix. Works fine for hierarchical structures.
- *Garbage collection*: storage isn't freed explicitly (using free operation), but rather implicitly: just delete pointers.
  - When the system needs storage, it searches through all of the pointers (must be able to find them all!) and collects things that aren't used.
  - If structures are circular then this is the only way to reclaim space.
  - Garbage collectors typically *compact* memory, moving objects to coalesce all free space.

- One way to implement garbage collection: *mark and sweep*:
  - Must be able to find all objects.
  - Must be able to find all pointers to objects.
  - Pass 1: mark. Go through all statically-allocated and procedure-local variables, looking for pointers. Mark each object pointed to, and recursively mark all objects it points to. The compiler has to cooperate by saving information about where the pointers are within structures.
  - Pass 2: sweep. Go through all objects, free up those that aren't marked.
- Garbage collection is often expensive: 10-20% or all CPU time in systems that use it.

STACK

What is a Stack?

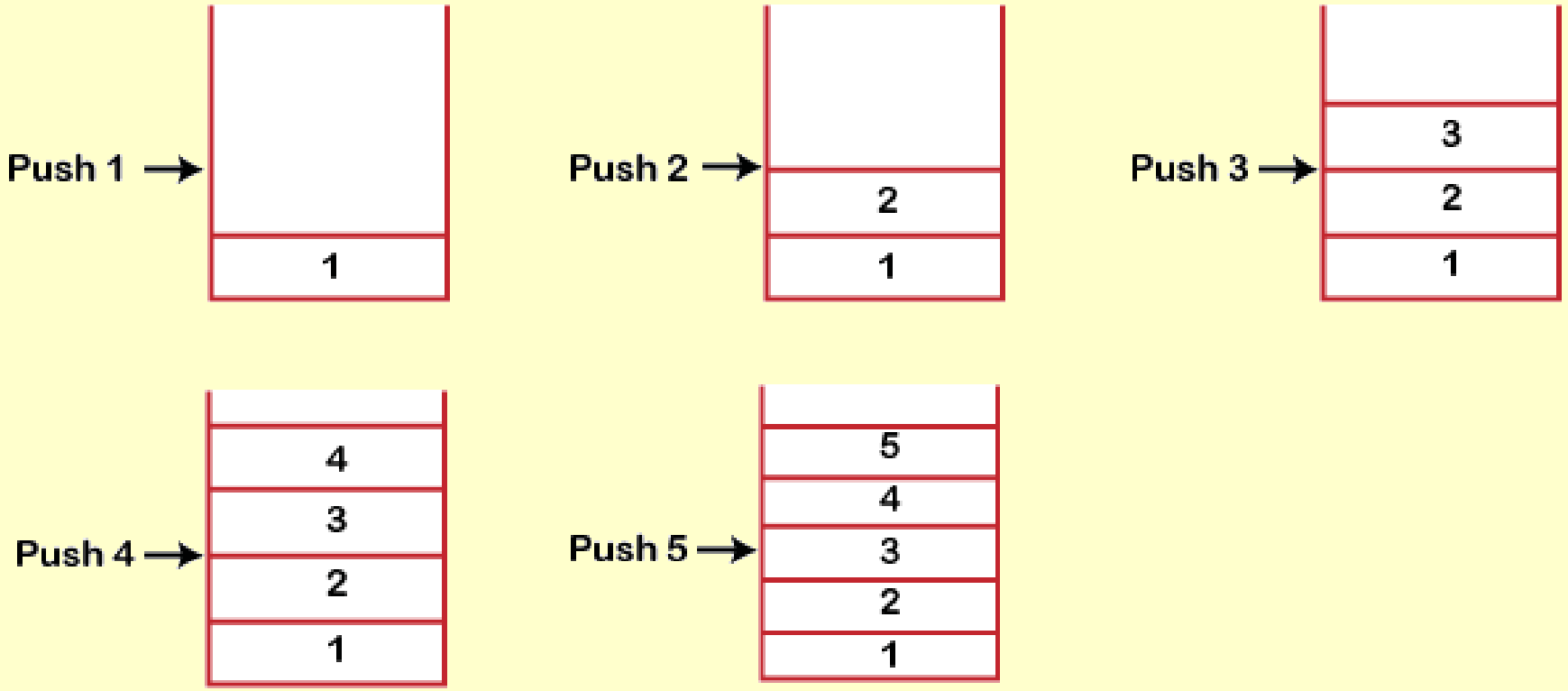
- A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle.
- Stack has one end, whereas the Queue has two ends (**front and rear**).
- It contains only one pointer **top pointer** pointing to the topmost element of the stack. Whenever an element is added in the stack,
- it is added on the top of the stack, and the element can be deleted only from the stack.
- In other words, a ***stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.***

Some key points related to stack

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
- A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
- It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO.

Working of Stack

- Stack works on the LIFO pattern.
- As we can observe in the below figure there are five memory blocks in the stack; therefore, the size of the stack is 5.
- Suppose we want to store the elements in a stack and let's assume that stack is empty.
- We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.


Push 1
1
Push 2
2
1
Push 3
3
2
1
Push 4
4
3
2
1
Push 5
5
4
3
2
1

- Since our stack is full as the size of the stack is 5.
- In the above cases, we can observe that it goes from the top to the bottom when we were entering the new element in the stack.
- The stack gets filled up from the bottom to the top.
- When we perform the delete operation on the stack, there is only one way for entry and exit as the other end is closed.
- It follows the LIFO pattern, which means that the value entered first will be removed last.
- In the above case, the value 5 is entered first, so it will be removed only after the deletion of all the other elements.



Standard Stack Operations

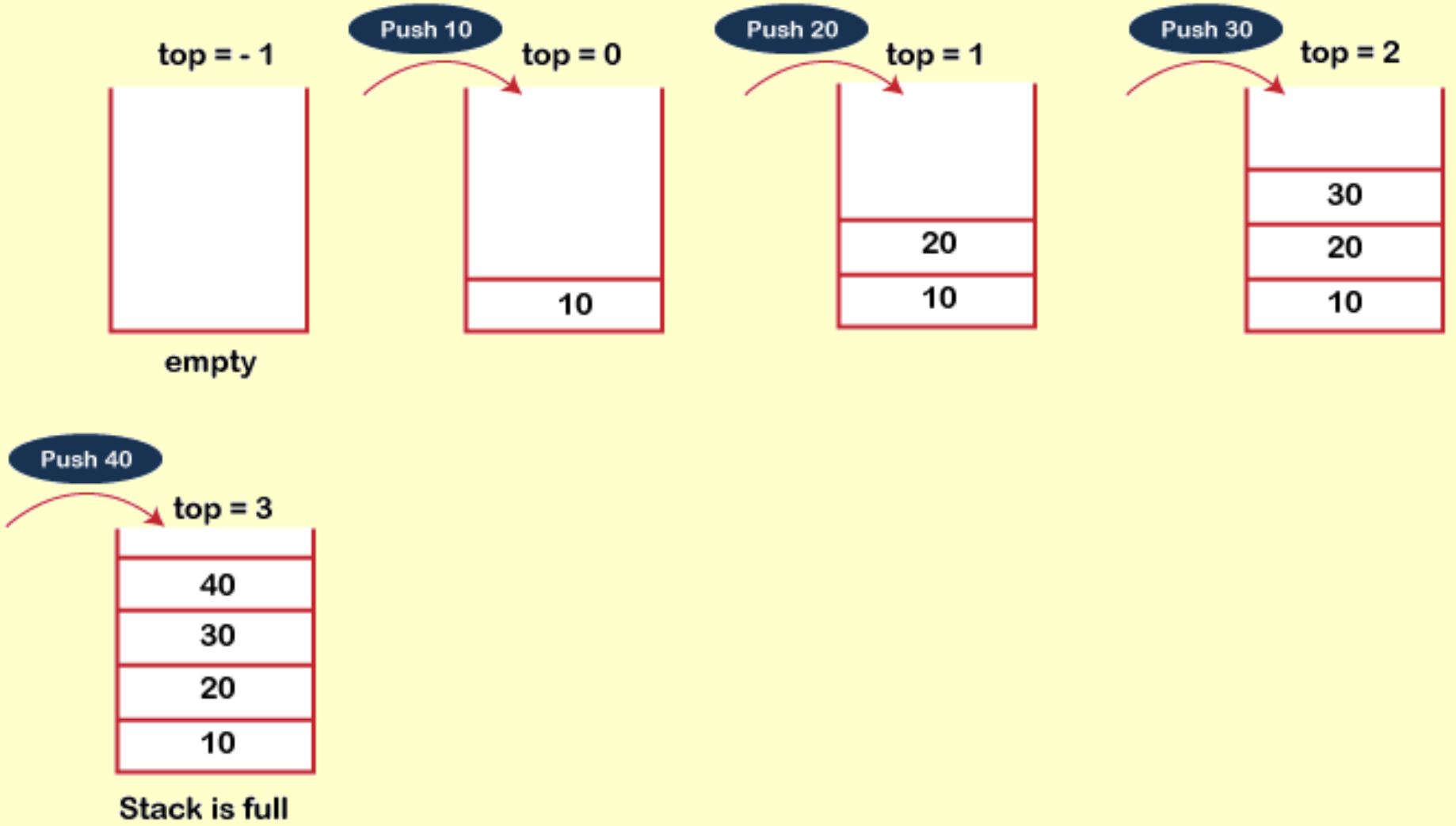
The following are some common operations implemented on the stack:

- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty():** It determines whether the stack is empty or not.
- **isFull():** It determines whether the stack is full or not.'
- **peek():** It returns the element at the given position.
- **count():** It returns the total number of elements available in a stack.
- **change():** It changes the element at the given position.
- **display():** It prints all the elements available in the stack.

PUSH operation

The steps involved in the PUSH operation is given below:

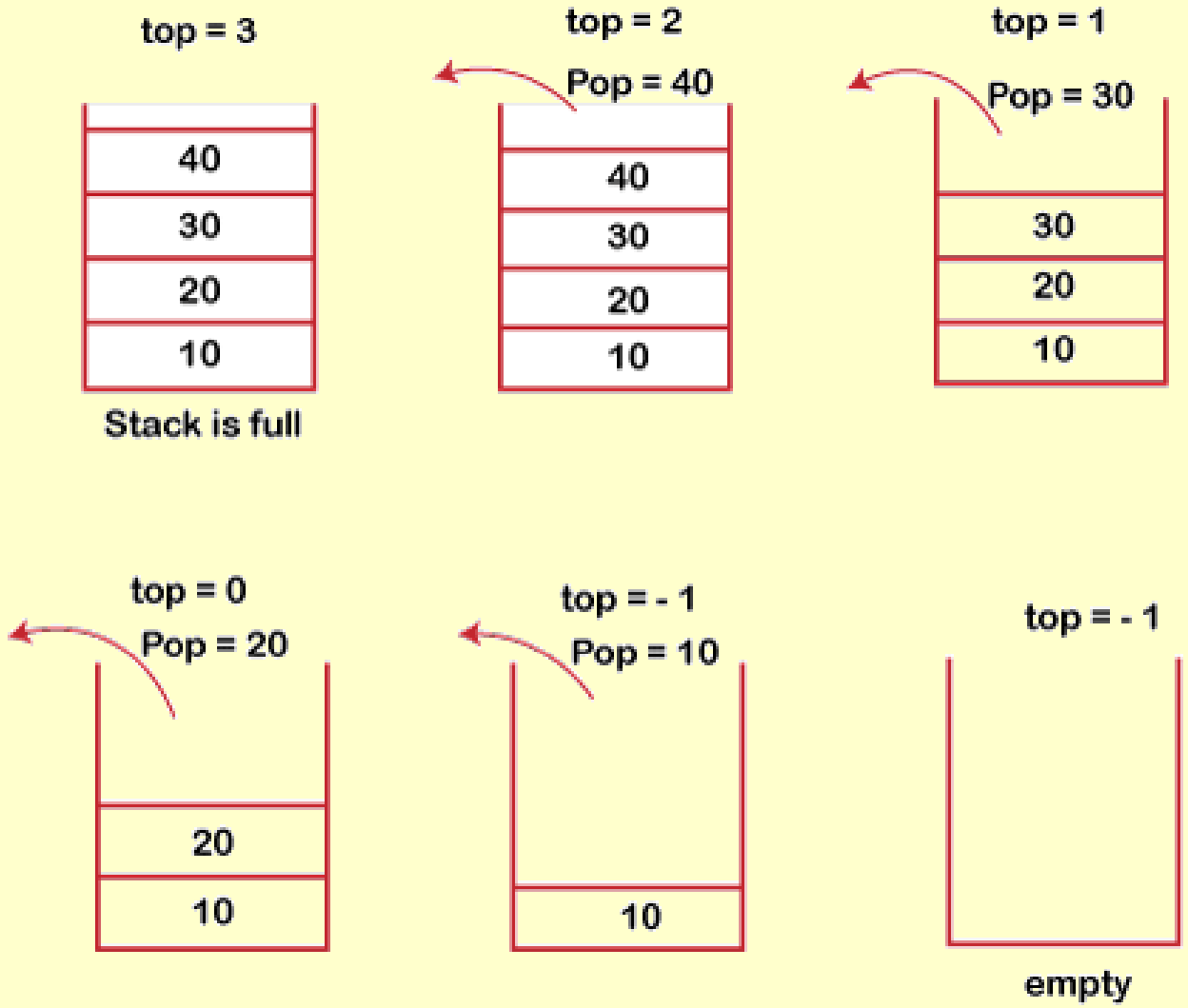
- Before inserting an element in a stack, we check whether the stack is full.
- If we try to insert the element in a stack, and the stack is full, then the ***overflow*** condition occurs.
- When we initialize a stack, we set the value of top as -1 to check that the stack is empty.
- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1,** and the element will be placed at the new position of the **top**.
- The elements will be inserted until we reach the ***max*** size of the stack.


top = - 1
empty
Push 10
top = 0
10
Push 20
top = 1
20
10
Push 30
top = 2
30
20
10
Push 40
top = 3
40
30
20
10
Stack is full

POP operation

The steps involved in the POP operation is given below:

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the ***underflow*** condition occurs.
- If the stack is not empty, we first access the element which is pointed by the ***top***
- Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1**.


top = 3
40
30
20
10
Stack is full
top = 2
Pop = 40
40
30
20
10
top = 1
Pop = 30
30
20
10
top = 0
Pop = 20
20
10
top = - 1
Pop = 10
10
top = - 1
empty

Applications of Stack

Balancing of symbols:

- Stack is used for balancing a symbol.
- For example, we have the following program:

```
int main()
{
   cout<<"Hello";
   cout<<"javaTpoint";
}
```

- As we know, each program has *an opening* and *closing* braces; when the opening braces come, we push the braces in a stack, and when the closing braces appear, we pop the opening braces from the stack.
- Therefore, the net value comes out to be zero. If any symbol is left in the stack, it means that some syntax occurs in a program.

String reversal:

- Stack is also used for reversing a string.
- For example, we want to reverse a "**javaTpoint**" string, so we can achieve this with the help of a stack.
- First, we push all the characters of the string in a stack until we reach the null character.
- After pushing all the characters, we start taking out the character one by one until we reach the bottom of the stack.

UNDO/REDO:

- It can also be used for performing UNDO/REDO operations.
- For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc. So, there are three states, a, ab, and abc, which are stored in a stack.
- There would be two stacks in which one stack shows UNDO state, and the other shows REDO state.
- If we want to perform UNDO operation, and want to achieve 'ab' state, then we implement pop operation.

Recursion:

- The recursion means that the function is calling itself again.
- To maintain the previous states, the compiler creates a system stack in which all the previous records of the function are maintained.

DFS(Depth First Search):

- This search is implemented on a Graph, and Graph uses the stack data structure.

Backtracking:

- Suppose we have to create a path to solve a maze problem.
- If we are moving in a particular path, and we realize that we come on the wrong way.
- In order to come at the beginning of the path to create a new path, we have to use the stack data structure.

Expression conversion:

- Stack can also be used for expression conversion.
- This is one of the most important applications of stack.
- The list of the expression conversion is given below:
    - Infix to prefix
    - Infix to postfix
    - Prefix to infix
    - Prefix to postfix
    - Postfix to infix

Memory management:

- The stack manages the memory.
- The memory is assigned in the contiguous memory blocks.
- The memory is known as stack memory as all the variables are assigned in a function call stack memory.
- The memory size assigned to the program is known to the compiler.
- When the function is created, all its variables are assigned in the stack memory.
- When the function completed its execution, all the variables assigned in the stack are released.

ARRAY IMPLEMENTATION OF STACK

- In array implementation, the stack is formed by using the array.
- All the operations regarding the stack are performed using arrays.
- Lets see how each operation can be implemented on the stack using array data structure.

Adding an element onto the stack (push operation)

- Adding an element into the top of the stack is referred to as push operation. Push operation involves following two steps.
- Increment the variable Top so that it can now refere to the next memory location.
- Add element at the position of incremented top. This is referred to as adding new element at the top of the stack.
- Stack is overflown when we try to insert an element into a completely filled stack therefore, our main function must always avoid stack overflow condition.

Algorithm:

- begin
- if top = n then stack full
- top = top + 1
- stack (top) : = item;
- end

Time Complexity : o(1)

Deletion of an element from a stack (Pop operation)

- Deletion of an element from the top of the stack is called pop operation. T
- he value of the variable top will be incremented by 1 whenever an item is deleted from the stack.
- The top most element of the stack is stored in an another variable and then the top is decremented by 1. the operation returns the deleted value that was stored in another variable as the result.
- The underflow condition occurs when we try to delete an element from an already empty stack.

Algorithm :

- begin
- **if** top = 0 then stack empty;
- item := stack(top);
- top = top - 1;
- end;

Time Complexity : o(1)

Visiting each element of the stack (Peek operation)

- Peek operation involves returning the element which is present at the top of the stack without deleting it.
- Underflow condition can occur if we try to return the top element in an already empty stack.

Algorithm :

- PEEK (STACK, TOP)
- Begin
- **if** top = -1 then stack empty
- item = stack[top]
- **return** item
- End

Time complexity: o(n)

```
#include <stdio.h>
int stack[100],i,j,choice=0,n,top=-1;
void push();
void pop();
void show();
void main ()
{
     printf("Enter the number of elements in the stack ");
    scanf("%d",&n);
    printf("*********Stack operations using array*********");
```

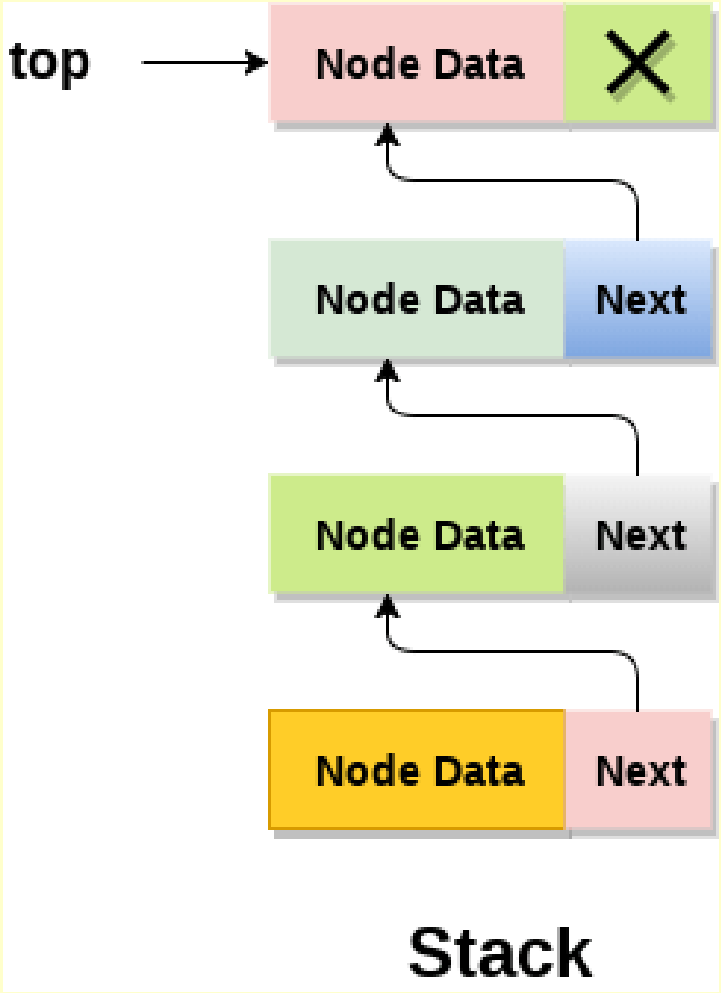
```
printf("\n----------------------------------------\n");
    while(choice != 4)
    {
        printf("Chose one from the below options...\n");
        printf("\n1.Push\n2.Pop\n3.Show\n4.Exit");
        printf("\n Enter your choice \n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
            {
                push();
                break;
            }
            case 2:
            {
                pop();
                break;
            }
            case 3:
            {
                show();
                break;
            }
            case 4:
            {
                printf("Exiting....");
                break;
            }
            default:
            {
                printf("Please Enter valid choice ");
            }
        };
    }
}
```

```
void push ()
{
    int val;
    if (top == n )
    printf("\n Overflow");
    else
    {
        printf("Enter the value?");
        scanf("%d",&val);
        top = top +1;
        stack[top] = val;
    }
}
```

```
void pop ()
{
    if(top == -1)
    printf("Underflow");
    else
    top = top -1;
}
void show()
{
    for (i=top;i>=0;i--)
    {
        printf("%d\n",stack[i]);
    }
    if(top == -1)
    {
        printf("Stack is empty");
    }
}
```

LINKED LIST IMPLEMENTATION OF STACK

- Instead of using array, we can also use linked list to implement stack.
- Linked list allocates the memory dynamically.
- However, time complexity in both the scenario is same for all the operations i.e. push, pop and peek.
- In linked list implementation of stack, the nodes are maintained non-contiguously in the memory.
- Each node contains a pointer to its immediate successor node in the stack.
- Stack is said to be overflown if the space left in the memory heap is not enough to create a node.
- The top most node in the stack always contains null in its address field.
- Lets discuss the way in which, each operation is performed in linked list implementation of stack.

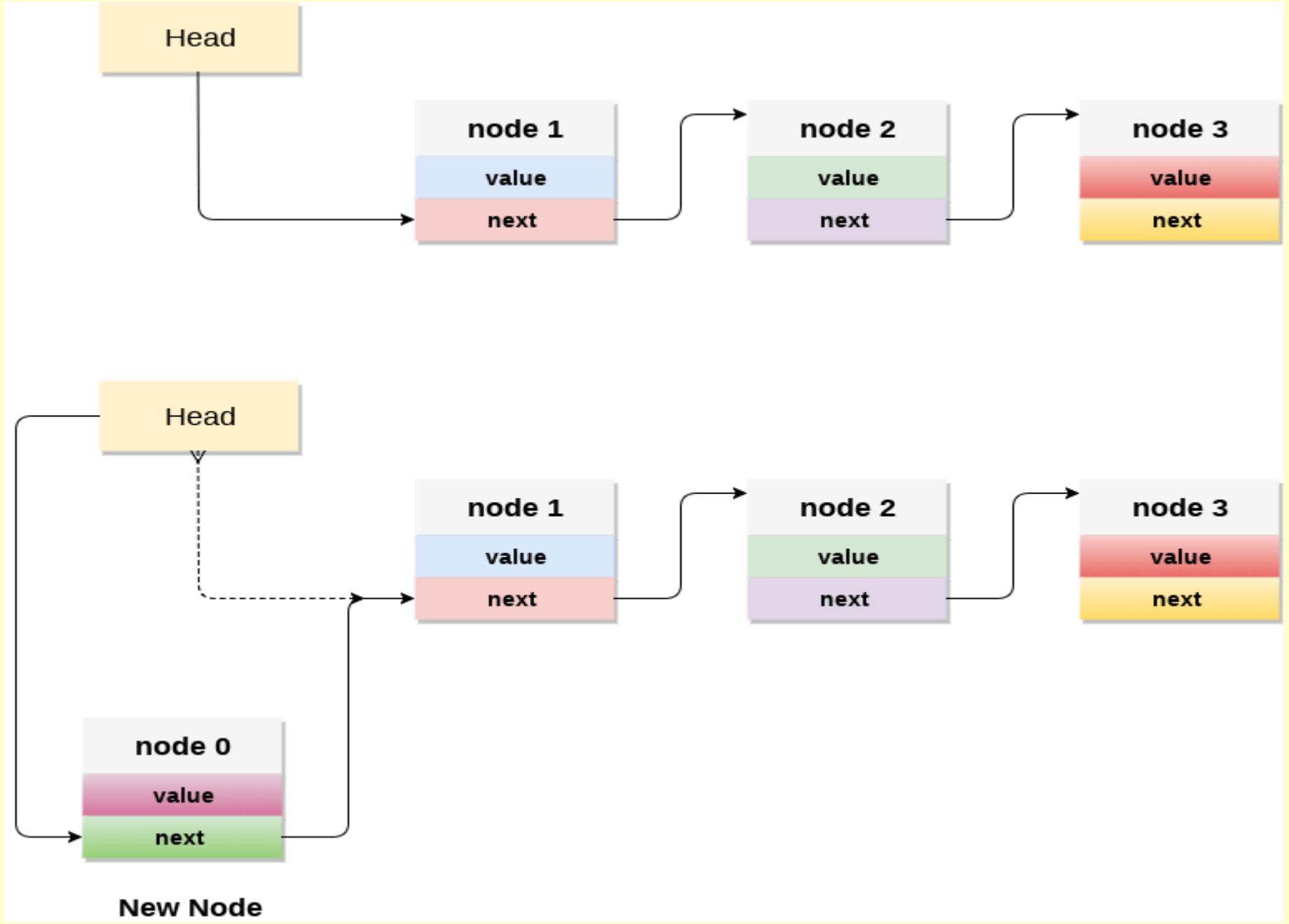

top
Node Data
Node Data
Next
Node Data
Next
Node Data
Next
Stack

Adding a node to the stack (Push operation)

- Adding a node to the stack is referred to as **push** operation.
- Pushing an element to a stack in linked list implementation is different from that of an array implementation.

In order to push an element onto the stack, the following steps are involved.

- Create a node first and allocate memory to it.
- If the list is empty then the item is to be pushed as the start node of the list. This includes assigning value to the data part of the node and assign null to the address part of the node.
- If there are some nodes in the list already, then we have to add the new element in the beginning of the list (to not violate the property of the stack). For this purpose, assign the address of the starting element to the address field of the new node and make the new node, the starting node of the list.
- **Time Complexity : o(1)**


Head
node 1
value
next
node 2
value
next
node 3
value
next
Head
node 1
value
next
node 2
value
next
node 3
value
next
node 0
value
next
New Node

Deleting a node from the stack (POP operation)

- Deleting a node from the top of stack is referred to as **pop** operation.
- Deleting a node from the linked list implementation of stack is different from that in the array implementation.
- In order to pop an element from the stack, we need to follow the following steps :
  - **Check for the underflow condition:** The underflow condition occurs when we try to pop from an already empty stack. The stack will be empty if the head pointer of the list points to null.
  - **Adjust the head pointer accordingly:** In stack, the elements are popped only from one end, therefore, the value stored in the head pointer must be deleted and the node must be freed. The next node of the head node now becomes the head node.
- **Time Complexity : o(n)**



Display the nodes (Traversing)

- Displaying all the nodes of a stack needs traversing all the nodes of the linked list organized in the form of stack.
- For this purpose, we need to follow the following steps.
  - Copy the head pointer into a temporary pointer.
  - Move the temporary pointer through all the nodes of the list and print the value field attached to every node.
- **Time Complexity : o(n)**

```c
#include <stdio.h>
#include <stdlib.h>
void push();
void pop();
void display();
struct node
{
int val;
struct node *next;
};
struct node *head;
void main ()
{
    int choice=0;
    printf("\n*********Stack operations using linked list*********\n");
    printf("\n----------------------------------------\n");
    while(choice != 4)
    {
        printf("\n\nChose one from the below options...\n");
        printf("\n1.Push\n2.Pop\n3.Show\n4.Exit");
        printf("\n Enter your choice \n");
        scanf("%d",&choice);
```

```
switch(choice)
 {
    case 1:
    {
        push();
        break;
    }
    case 2:
    {
        pop();
        break;
    }
    case 3:
    {
        display();
        break;
    }
    case 4:
    {
        printf("Exiting....");
        break;
    }
    default:
    {
        printf("Please Enter valid choice ");
    }
  };
}
}
```

```
void push ()
{
    int val;
    struct node *ptr = (struct node*)malloc(
sizeof(struct node));
    if(ptr == NULL)
    {
        printf("not able to push the element");

    }
    else
    {
        printf("Enter the value");
        scanf("%d",&val);
    if(head==NULL)
        {
            ptr->val = val;
            ptr -> next = NULL;
            head=ptr;
        }
        else
        {
            ptr->val = val;
            ptr->next = head;
            head=ptr;
        }
        printf("Item pushed");
    }
}
```

```
void pop()
{
    int item;
    struct node *ptr;
    if (head == NULL)
    {
        printf("Underflow");
    }
    else
    {
        item = head->val;
        ptr = head;
        head = head->next;
        free(ptr);
        printf("Item popped");

    }
}
```

```
void display()
{
    int i;
    struct node *ptr;
    ptr=head;
    if(ptr == NULL)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Printing Stack elements \n");
        while(ptr!=NULL)
        {
            printf("%d\n",ptr->val);
            ptr = ptr->next;
        }
    }
}
```

